

ECHOCITY

A Living AI Civilization Running Completely On-Device

Architectural Specification & Production-Ready Software Design Document

CORE ARCHITECTURE MANDATE

Zero cloud dependencies. Zero external inference pipelines. Complete offline execution via local context assembly, embedded vector indices, asynchronous state cycles, and highly optimized quantization boundaries on local edge consumer hardware.

DOCUMENT REF	SDD-ECHOCITY-2026-V1
AUTHOR STATUS	Lead Systems Architect / PM / Technical Writer Matrix
TARGET STACK	FastAPI / Canvas API / PostgreSQL / Ollama / Docker Compose
DATE OF ISSUE	July 3, 2026

TABLE OF CONTENTS

1. Executive Summary & Vision Matrix	3
2. System Feature Breakdown & Capabilities	4
3. AI Citizen Micro-Architecture	7
4. Agent Thinking Cycle Engine	9
5. World Simulation Engine & Spatio-Temporal Loop	11
6. High-Level Software Architecture	13
7. Frontend Architecture & Canvas Subsystems	15
8. Backend Service Layer & Async Core	17
9. Relational Database Schema Design (PostgreSQL)	19
10. Local AI Inference Infrastructure	22
11. Infrastructure, Containerization & Deployment	24
12. Repository Topography & Structural Architecture	26
13. High-Velocity Hackathon Roadmap & Milestones	28
14. Strategic Pitch Strategy & Judge Alignment	30
15. Open Source Evolution & Scalability Vectors	32
16. Strategic Risk Management Matrix	34

1. EXECUTIVE SUMMARY & VISION MATRIX

1.1 Paradigmatic Shift in Simulative Entertainment

Modern gaming architectures rely heavily on deterministic state machines, branch-and-bound behavior trees, or pre-scripted dialogue branches written by narrative teams. While these architectures create a polished illusion of choice, they fundamentally fail to deliver an organic, reactive world. **EchoCity** replaces scripted narrative structures with an emergency-driven, fully autonomous AI simulation running completely on consumer-grade edge hardware.

1.2 Complete Strategic Mandate: On-Device Integrity

The defining constraint and competitive edge of EchoCity is its absolute independence from remote cloud networks. By combining high-performance local LLM backends (Ollama running specialized small language models like Gemma, Qwen, or Phi), explicit transactional databases, and localized asynchronous vector spaces, EchoCity operates fully disconnected from the internet. This design resolves critical modern development blockers:

- **Zero API Operational Overhead:** Eliminates vulnerable runtime cost structures driven by per-token cloud pricing patterns.
- **Deterministic Sub-millisecond Network Latency:** Local IPC loopbacks guarantee the system bypasses external TCP/IP transit delays.
- **Absolute Data Isolation:** Complete simulation logs, personal agent dynamics, and operational context trees live and die within the local client stack.

THE PLAYER'S ROLE

The player does not drive the narrative arc; the system drives itself. The player chooses a systemic vocation (Detective, Judge, Investigator, or Mediator) and interfaces directly with the emergent realities of the world via natural language queries, acting as an anomaly within a self-sustaining societal simulator.

2. SYSTEM FEATURE BREAKDOWN & CAPABILITIES

| 2.1 Core Architectural Features

Living AI Citizens

Every non-player character (NPC) within the system layout is modeled as an independent, stateful entity executing a persistent loop of observation, reflection, and plan formulation. They possess distinct socioeconomic properties, biological states, and structural schedules.

Persistent Memory & Vector Retrieval Spaces

Rather than utilizing flat key-value state tables, memories are layered through a hybrid transactional storage layer (PostgreSQL) and a local contextual embedding space. Ephemeral short-term event records are continuously synthesized, condensed, and transferred into deep semantic memory clusters.

Dual-Axis Emotion Engine

Dynamic behavioral states are mathematically managed via continuous floating-point values tracking key indices: **Trust**, **Fear**, **Greed**, **Stress**, and **Morality**. These attributes modulate prompt templates dynamically, shifting sentence patterns, lexical selection, and action probability tables.

The Relationship Graph Topology

Societal interaction matrices are structured as an active mathematical graph where nodes represent individual citizens and edges carry dynamic multi-dimensional weights defining structural bonds (e.g., Affection, Suspicion, Business Obligations). These edges change continuously based on transactional event logs.

Asynchronous Internal Monologue Core

Before any outward communication or action execution occurs, the system triggers an internal thought tick. The agent parses its current contextual frame to write an un-redacted, hidden analytical assessment of its environment, immediate threats, and personal agendas.

Automated Task Schedulers & Goal Planners

A deterministic chronographic system emits periodic ticks causing characters to review their active agenda. Agents prioritize high-level long-term goals (e.g., "Pay off debt to the black market merchant") and translate them into granular sequential task buckets handled by the Daily Scheduler.

Dynamic Crime & Evidence Generation Pipeline

Crimes are not randomized event occurrences. They emerge organically from alignment anomalies: an agent experiencing elevated **Stress** combined with low **Morality** and high **Greed** targets an edge in the Relationship

Graph. The execution mechanics leave spatial logs (e.g., dropped objects, timestamp discrepancies) stored in the centralized Evidence System.

Systemic Features Reference Blueprint

System Module	Underlying Architecture	Data Transaction Pattern
Rumor Engine	Graph-based token decay distribution. Bits of truth mutate based on agent trust matrices.	Pub/Sub mutation via Event Bus.
News Broadcast	Automated chronographic parser compiling evidence tables into human-readable bulletins.	Scheduled ingestion of world state.
Dynamic Economy	Double-entry ledger mapping finite coin/resource distribution among citizens.	ACID-compliant PostgreSQL transactions.
Investigation UI	Custom Canvas interactive layout showing cross-referenced data grids.	Bi-directional WebSocket streaming.

3. AI CITIZEN MICRO-ARCHITECTURE

3.1 Complete State Model Design

Every individual agent is modeled as an atomic data cluster and a behavioral runtime engine. The schema definition encapsulates deterministic metrics alongside abstract neural contexts, ensuring behavioral stability and operational predictability across thousands of execution cycles.

```
{
  "agent_id": "UUIDv4",
  "identity": {
    "name": "Marcus Vance",
    "age": 34,
    "occupation": "Sub-level Grid Technician",
    "beliefs": "Radical anti-automation, devout preservationist"
  },
  "psychometric_matrix": {
    "morality": 0.32,
    "greed": 0.85,
    "stress": 0.68,
    "fear": 0.41,
    "energy": 0.90
  },
  "operational_states": {
    "current_task": "Siphon plasma from Main Conduit B",
    "hidden_secrets": ["Embezzled 400 credits from Sector 4 logistics fund"],
    "financial_status": {"credits": 142.50}
  }
}
```

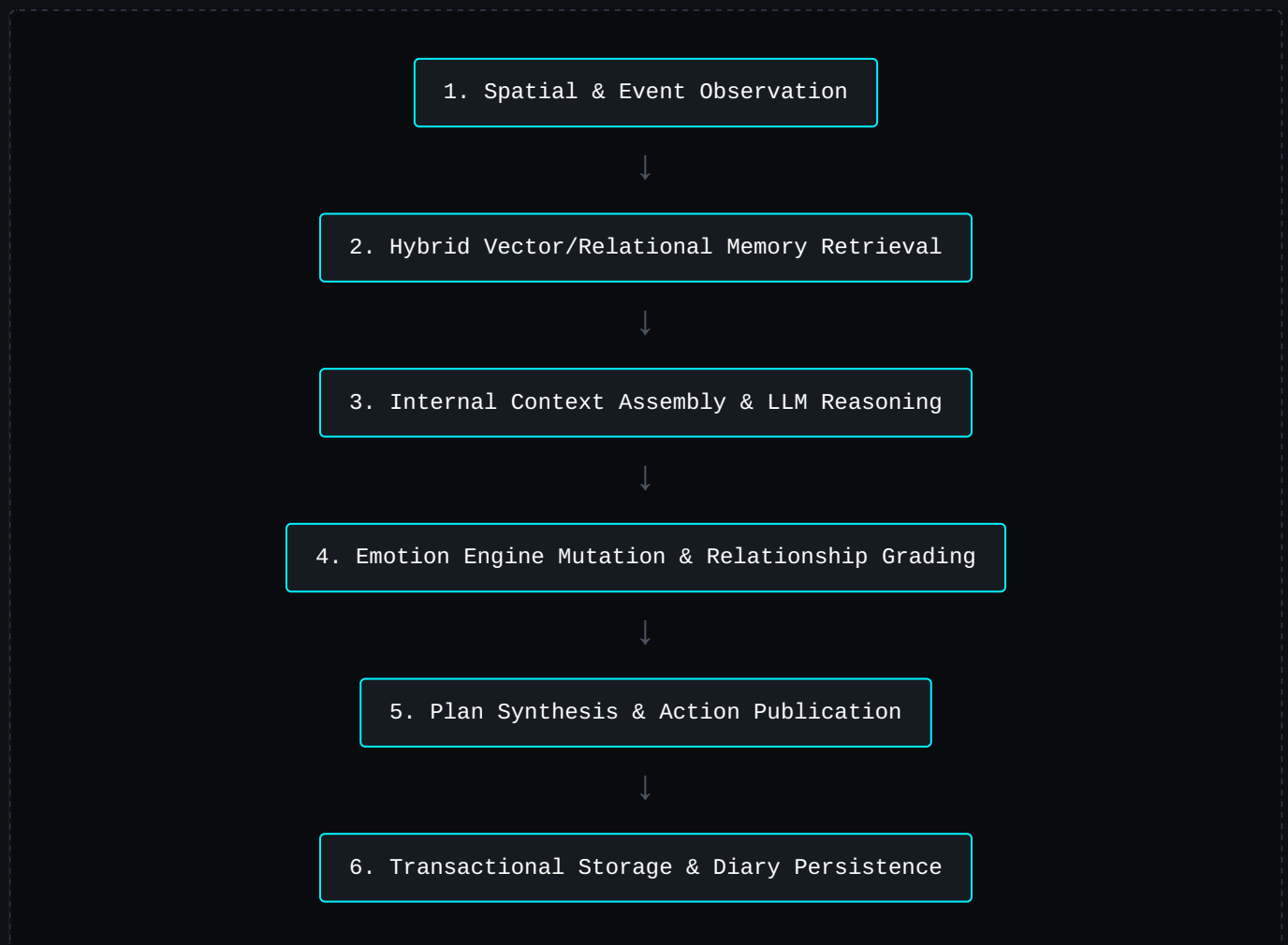
3.2 Behavioral Regulators & Cognitive Variables

- **Morality (0.00 - 1.00):** Regulates the compliance boundary of generated plans. Agents with low morality ignore city laws when selecting goals under high financial strain.
- **Greed (0.00 - 1.00):** Modulates transaction thresholds. Influences negotiation responses, asset collection behavior, and vulnerability to bribery vectors.
- **Stress (0.00 - 1.00):** Exponentially reduces the reasoning window depth during local LLM prompt construction, causing fragmented dialogue choices and error-prone tactical plans.

4. AGENT THINKING CYCLE ENGINE

4.1 Continuous Cognitive Pipeline Architecture

The thinking cycle is an infinite state loop executed on dedicated background workers within the FastAPI engine layer. It ensures that every action taken by an agent is grounded in history, emotion, and explicit rationale.



4.2 Mechanical Stage Deconstruction

Step 1: Observe: The worker scans the database for spatial state changes, active environment events, and incoming messages targeting the agent's spatial quadrant.

Step 2: Retrieve Memories: A query is dispatched to the hybrid storage architecture. It constructs a context window by retrieving the top 5 relevant long-term memories via Cosine Similarity from the vector database, along with recent interaction histories from PostgreSQL.

Step 3: Reason & Plan: The gathered data is injected into a highly structured markdown template. The local LLM parses the prompt and generates a structured JSON output outlining internal reasoning, dialogues, and immediate environment manipulations.

Step 4: Execute & Publish: The parsed actions are validated against the global game state rules. If approved, they are committed to the centralized database and dispatched across the application Event Bus to update listening frontends in real-time.

5. WORLD SIMULATION ENGINE & SPATIO-TEMPORAL LOOP

| 5.1 Deterministic Simulation Ticks

To avoid race conditions and resource starvation on local hardware, EchoCity relies on a centralized simulation clock. The world tracks time through discrete atomic units called **Simulation Ticks**. By default, 1 Tick represents 15 minutes of simulated time, executing every 5 seconds in real-world time during accelerated operation mode.

| 5.2 Environmental & Societal Cascades

The system orchestrates interactions across multiple overlapping layers:

- **Weather Subsystem:** Periodically mutates environmental vectors (e.g., Acid Rain, Thick Smog). Severe weather shifts agent paths, forces indoor scheduling patterns, and modifies detection variables within the Crime Engine.
- **Rumor Spread Pipeline:** When an agent witnesses a major event or uncovers a secret, the event is saved as a localized rumor block. When interacting with trusted adjacent graph nodes, the rumor is selectively shared, losing fidelity and gaining malicious mutations based on the intermediate agents' **Greed** or **Fear** vectors.

6. HIGH-LEVEL SOFTWARE ARCHITECTURE

6.1 Clean Architecture Topology

The system implements an offline-first Clean Architecture pattern. The layout enforces strict decoupling between the client UI view layer, the core simulation processing loop, and the local model execution framework.

[Image of standard Clean Architecture layered concentric circles showing UI, Controllers, Use Cases, and Entities]

6.2 System Component Relationships

Component Block	Primary Framework	Architectural Isolation Rule
Presentation View Layer	Vanilla HTML5 / HTML5 Canvas API	Zero knowledge of internal database structures; communicates purely through JSON over WebSockets/REST.
Simulation Engine Core	Python Asynchronous Workers	Manages scheduling metrics, tick distributions, and multi-agent state reconciliation.
Model Inference Node	Ollama IPC Interface	Stateless prompt-to-token computation engine. Isolated from direct database queries.

7. FRONTEND ARCHITECTURE & CANVAS SUBSYSTEMS

| 7.1 Non-Framework Mechanical Implementation

To meet the extreme constraints of a hackathon timeline and prevent framework build performance blocks, the user interface is designed as an optimized, single-page application built on pure HTML5, CSS3, and an interactive 2D Canvas rendering grid. This guarantees sub-millisecond drawing routines and complete elimination of complex NPM dependency configurations.

| 7.2 Screen Layout and Interactive Modules

- **City Map Navigation Grid:** An HTML5 Canvas view drawing a real-time matrix of the city zones, tracking individual agent avatars moving between structures according to active schedules.
- **Investigation Dashboard & Evidence Board:** A visual viewport where users map spatial evidence node links, trace relationship anomalies, review interrogation records, and build timeline flows.

8. BACKEND SERVICE LAYER & ASYNC CORE

| 8.1 FastAPI Architecture Layout

The application engine uses **FastAPI** powered by an asynchronous execution worker pool (Uvicorn). Asynchronous route endpoints handle user interactions, while non-blocking background loops run the infinite execution loops for the AI agents.

| 8.2 Core Endpoints Specifications

GET /api/v1/simulation/state: Returns the active tick configuration, weather state, and global overview statistics.

POST /api/v1/interrogate/{agent_id}: Channels player input text directly into the conversation evaluation loop, returning an expressive text response and updating the target agent's suspicion value.

WS /api/v1/stream/events: Bi-directional pipeline streaming state updates, agent transitions, and emergent crime records to the UI client interface.

9. RELATIONAL DATABASE SCHEMA DESIGN (POSTGRESQL)

9.1 Entity-Relationship Layout Data Definition Language

The system relies on an explicit, ACID-compliant structure within PostgreSQL to safeguard transaction histories across complex state cascades.

```
CREATE TABLE agents (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(100) NOT NULL,  
  occupation VARCHAR(100) NOT NULL,  
  morality NUMERIC(3,2) CHECK (morality >= 0.00 AND morality <= 1.00),  
  greed NUMERIC(3,2) CHECK (greed >= 0.00 AND greed <= 1.00),  
  stress NUMERIC(3,2) CHECK (stress >= 0.00 AND stress <= 1.00),  
  credits NUMERIC(12,2) DEFAULT 0.00,  
  current_quadrant_x INT,  
  current_quadrant_y INT  
);  
  
CREATE TABLE relationships (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  source_agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,  
  target_agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,  
  trust_level NUMERIC(3,2) DEFAULT 0.50,  
  suspicion_level NUMERIC(3,2) DEFAULT 0.00,  
  UNIQUE(source_agent_id, target_agent_id)  
);  
  
CREATE TABLE memories (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  agent_id UUID REFERENCES agents(id) ON DELETE CASCADE,  
  tick_recorded INT NOT NULL,  
  raw_content TEXT NOT NULL,  
  embedding_vector VECTOR(384),  
  is_long_term BOOLEAN DEFAULT FALSE  
);
```

9.2 Schema Relations Framework

The `memories` and `relationships` tables map directly to the primary `agents` records via foreign key constraints. The system leverages pgvector to handle quick semantic matches directly alongside standard transactional database fields.

10. LOCAL AI INFERENCE INFRASTRUCTURE

10.1 Absolute Edge Deployment Topology

EchoCity bypasses external cloud systems by deploying **Ollama** as a local background inference service on the user's desktop hardware. By binding to local inference runtimes optimized for CPU/GPU compilation patterns, the system matches high-quality responses with predictable execution loops.

[Image of the Local Context Assembly Engine detailing Vector Search, Relational Context injection, and Prompt Construction]

10.2 Context Management & Prompt Formats

To extract reliable, structured data profiles from highly quantized 4-bit models (e.g., **qwen2.5:7b-instruct-q4_K_M**), prompts are packaged in strict XML block configurations that force JSON outputs:

```
<system>
You are running the core brain partition for Marcus Vance.
Current Psychometrics: Morality=0.32, Greed=0.85, Stress=0.68.
Retrieved Memories: "Needs 400 credits to cover black market loan debts."
Respond ONLY as a valid JSON block containing:
{ "thought": "internal monologue", "dialogue": "outward speech", "action": "system_command" }
</system>
```

11. INFRASTRUCTURE, CONTAINERIZATION & DEPLOYMENT

11.1 Complete Multi-Container Configuration

The environment is managed via a single `docker-compose.yml` file, packaging the isolated runtime systems into clean networks that interact via high-speed local ports.

```
version: '3.8'
services:
  database:
    image: ankane/pgvector:latest
    environment:
      POSTGRES_DB: echocity_isolated
      POSTGRES_PASSWORD: terminal_secret_pass
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  ollama-node:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_storage:/root/.ollama

  backend-engine:
    build: ./backend
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000
    ports:
      - "8000:8000"
    depends_on:
      - database
      - ollama-node
    environment:
      - DATABASE_URL=postgresql://postgres:terminal_secret_pass@database:5432/echocity_isolated
      - OLLAMA_HOST=http://ollama-node:11434
```

12. REPOSITORY TOPOGRAPHY & STRUCTURAL ARCHITECTURE

12.1 Monorepo Topography Design

The development repository is configured as a clear monorepo workspace to ensure zero configuration friction and isolated modules during rapid hackathon builds.

```
.
├── .github/           # CI/CD validation automation
├── apps/
│   └── web-client/    # SPA HTML5/CSS3/Canvas core code
├── engine/
│   ├── ai/           # Prompt structures and Ollama controllers
│   ├── simulation/    # Tick execution loops and systems trackers
│   └── database/      # PostgreSQL connection and migrations
├── docker/           # Deployment assets and volume configurations
└── scripts/          # Automated setup and model pulling utils
```


13. HIGH-VELOCITY HACKATHON ROADMAP & MILESTONES

13.1 5-Day Accelerated Execution Matrix

Timeline Phase	Target Goals	Core Verification Deliverables
Day 1	Core Infrastructure Buildout	Docker multi-container platform functional; pgvector verification queries returning matching vectors.
Day 2	Agent Cognition Loop Engineering	Background loops executing agent prompt assemblies and writing data to local memory tables.
Day 3	Frontend View Implementation	Canvas layer drawing active agent movements; live WebSocket server streaming state ticks.
Day 4	Systemic Features Integration	Crime Engine processing events; News feeds updating via state triggers.
Day 5	Optimization & Polish Phase	Offline demo testing with local Wi-Fi disconnected; presentation recording.

14. STRATEGIC PITCH STRATEGY & JUDGE ALIGNMENT

| 14.1 Key Strategic Pitch Angle

The core presentation strategy targets the main limitation of cloud systems: "Why does every AI game feel like an expensive chatbot web app?" The response highlights our architecture's strengths:

- **Absolute Resource Independence:** Disconnect the demo machine from the network entirely to showcase local execution.
- **Emergent Narrative Highlights:** Show the judges the underlying data tables where an agent commits a crime driven by its unique financial profile, completely free of hardcoded script configurations.

15. OPEN SOURCE EVOLUTION & SCALABILITY VECTORS

| 15.1 Modular Civilizational Roadmap

Beyond the hackathon, EchoCity is designed to scale into an open-source, multi-agent sandbox framework. The architectural decoupled layout allows community developers to easily drop in new modules:

- **Political Subsystems:** Agents running for civic seats, running local campaigns, and altering tax matrices based on their **Greed** states.
- **Socio-Economic Lineages:** Support for agent lineages, legacy wealth distribution structures, and dynamic small-business creation pipelines.

16. STRATEGIC RISK MANAGEMENT MATRIX

16.1 Systemic Risk Mitigation Profiling

Identified Risk Vector	Operational Impact	Architectural Resolution Plan
Local Model Hallucinations	High. Agents may output invalid JSON or break simulation formatting rules.	Enforce hard structural checks in the parsing code. Fall back to fallback behavioral templates if validation fails.
Hardware Bottlenecks	Critical. High CPU load can stall ticks and freeze the user interface.	Decouple the backend worker loop into a separate sequence. Limit active LLM updates to nearby agents.